Spring Boot Auto-configuration & Actuator

Opinionated defaults, zero-XML setup, production-ready monitoring

Sub-Chapter 03-05 | Beginner-Friendly | With Diagrams & Q&A;

What is Spring Boot?	Opinionated framework: embedded server, no XML
@SpringBootApplication	@Configuration + @ComponentScan + @EnableAutoConfiguration
How Auto-configuration Works	spring.factories / @Conditional drives bean creation
application.properties / yml	Central place for all app settings
Profiles	@Profile & spring.profiles.active for env switching
Spring Boot Actuator	Health, info, metrics endpoints out of the box
Custom Auto-configuration	Write your own starter with @Conditional

1 What is Spring Boot?

Opinionated framework: embedded server, zero XML

Overview

Spring Boot is a framework built on top of Spring that lets you create stand-alone, production-ready applications with **almost no configuration**. It follows **convention over configuration**: it picks sensible defaults for you, so you only override what you actually need to change. An embedded server (Tomcat by default) is bundled inside your JAR, so you just run **java -jar myapp.jar** -- no separate deployment needed.

Simple Java Example

```
// pom.xml parent: spring-boot-starter-parent
// dependency:    spring-boot-starter-web

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;

@SpringBootApplication
public class HelloApp {
    public static void main(String[] args) {
        SpringApplication.run(HelloApp.class, args);
    }
}

@RestController
class HelloController {
    @GetMapping("/hello")
    public String hello() { return "Hello, Spring Boot!"; }
}
```

Key Rules

- Spring Boot auto-configures components based on what is on the classpath.
- The embedded Tomcat runs on port 8080 by default -- override with server.port.
- 'Convention over configuration' means less code for common setups.
- spring-boot-starter-* POMs pull in all needed dependencies for a feature.
- No web.xml, no applicationContext.xml needed -- Java config only.

Q1. What is the main benefit of Spring Boot over plain Spring?

Spring Boot removes boilerplate configuration. You get auto-configured beans, an embedded server, and starter POMs -- all out of the box.

Q2. What does 'convention over configuration' mean?

The framework applies popular defaults automatically. You only provide configuration when your requirements differ from those defaults.

Q3. How do you run a Spring Boot application?

Run 'mvn spring-boot:run' during development or 'java -jar app.jar' for the packaged fat JAR.

Q4. What is a fat JAR?

A JAR that contains the application code plus all dependency JARs and the embedded server, making it completely self-contained.

Q5. What port does Spring Boot use by default?

Port 8080. Change it with 'server.port=9090' in application.properties.

2

@SpringBootApplication

@Configuration + @ComponentScan + @EnableAutoConfiguration

What Does @SpringBootApplication Do?

@SpringBootApplication is a **convenience annotation** that combines three annotations into one. You almost always put it on your main class.

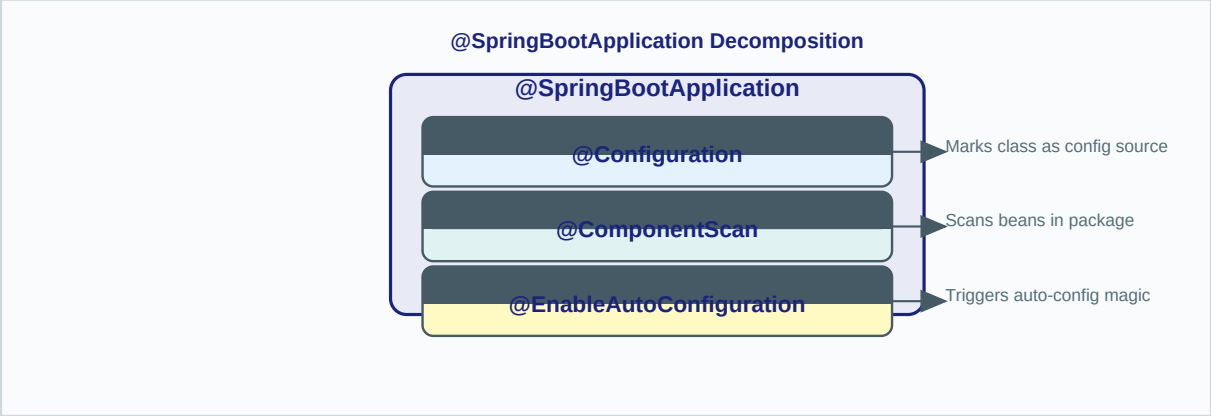

Figure: @SpringBootApplication is shorthand for three annotations

Simple Java Example

```
// These two are IDENTICAL:

// Option A -- the shortcut
@SpringBootApplication
public class MyApp { ... }

// Option B -- the explicit form
@Configuration
@ComponentScan(basePackages = "com.example")
@EnableAutoConfiguration
public class MyApp { ... }
```

Key Rules

- @Configuration: marks the class as a source of @Bean definitions.
- @ComponentScan: scans the current package and sub-packages for Spring components.
- @EnableAutoConfiguration: tells Spring Boot to auto-configure based on classpath.
- Place @SpringBootApplication only on the main class in the root package.
- Sub-packages are scanned automatically -- no need to list them.

Q1. What three annotations does @SpringBootApplication replace?

@Configuration, @ComponentScan, and @EnableAutoConfiguration.

Q2. Why is @ComponentScan important?

It tells Spring where to look for beans. By default it scans the package of the annotated class and all sub-packages.

Q3. Can I use @SpringBootApplication on any class?

Technically yes, but it should go on the main class in the root package so component scanning covers your entire application.

Q4. What happens if I put my main class in a sub-package?

Component scanning may miss beans in sibling or parent packages. Always keep the main class in the root package.

Q5. Does @SpringBootApplication make the class a Spring bean?

Yes -- because it includes @Configuration, which is itself a @Component, so Spring manages it.

3 How Auto-configuration Works

spring.factories / AutoConfiguration.imports + @Conditional

The Mechanism

When your application starts, @EnableAutoConfiguration reads a special file inside spring-boot-autoconfigure.jar. In Spring Boot 2 this file is **META-INF/spring.factories**; in Spring Boot 3 it is **META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports**. Each entry is a candidate auto-configuration class. Spring Boot then evaluates **@Conditional** annotations on each class -- if conditions pass, the beans are registered; otherwise they are silently skipped.

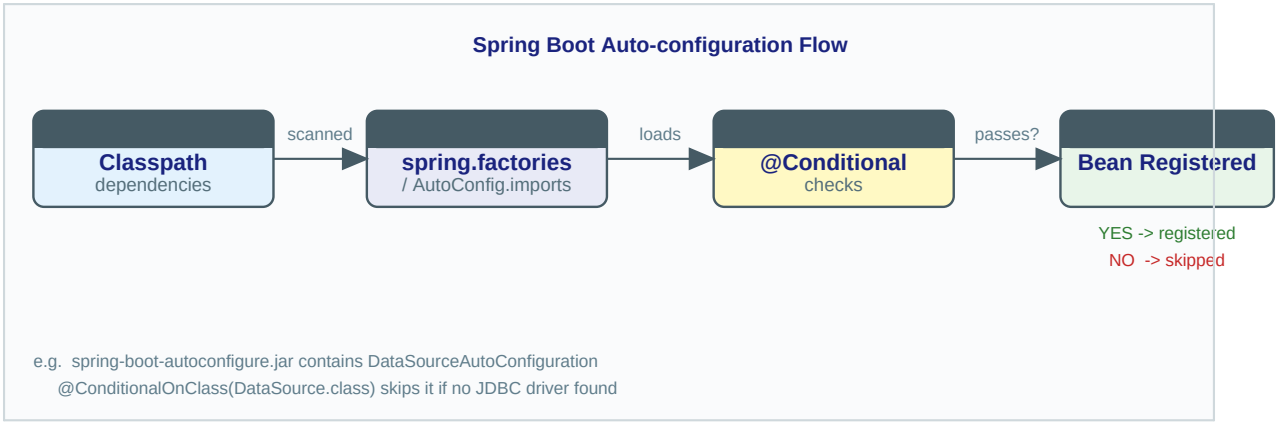

Figure: Auto-config flow from classpath scan to bean registration

Simple Java Example -- Custom Condition

```
// Auto-config class (simplified)
@Configuration
@ConditionalOnClass(DataSource.class)          // only if JDBC driver present
@ConditionalOnMissingBean(DataSource.class)    // only if app has no custom one
public class DataSourceAutoConfiguration {

    @Bean
    public DataSource dataSource() {
        return new HikariDataSource();         // sensible default
    }
}
```

Key Rules

- @ConditionalOnClass: register bean only if a class is on the classpath.
- @ConditionalOnMissingBean: register bean only if user has NOT defined one already.
- @ConditionalOnProperty: register bean only if a property is set to a certain value.
- User-defined beans always take priority over auto-configured beans.
- Run with --debug flag to see a full auto-configuration report in the console.

Q1. Where does Spring Boot find auto-configuration candidates?

In META-INF/spring.factories (Boot 2) or META-INF/spring/...AutoConfiguration.imports (Boot 3) inside the autoconfigure JAR.

Q2. What is @ConditionalOnClass?

It tells Spring Boot to create the annotated bean only when a specific class exists on the classpath.

Q3. Can I disable a specific auto-configuration?

Yes -- use @SpringBootApplication(exclude = DataSourceAutoConfiguration.class) or set spring.autoconfigure.exclude in application.properties.

Q4. Do auto-configured beans override my custom beans?

No. Your beans always win. @ConditionalOnMissingBean ensures auto-config backs off.

Q5. How can I see which auto-configurations were applied?

Start the app with --debug and look for the 'CONDITIONS EVALUATION REPORT' in the log.

4 application.properties / application.yml

Central configuration for your Spring Boot app

Overview

Spring Boot reads **src/main/resources/application.properties** (or the YAML equivalent **application.yml**) at startup. Every property you set there overrides the corresponding auto-configuration default. You can also inject values directly into your beans with **@Value** or bind a whole prefix to a class with **@ConfigurationProperties**.

application.properties example

```
# application.properties
server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
app.greeting=Hello from config!
```

application.yml equivalent

```
# application.yml
server:
  port: 8081
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: secret
app:
  greeting: Hello from config!
```

Injecting values into a bean

```
@Component
public class GreetingService {

    @Value("${app.greeting}")
    private String greeting;

    public void greet() {
        System.out.println(greeting);
    }
}
```

Key Rules

- application.properties and application.yml are interchangeable -- pick one style.
- @Value("${key}") injects a single property; @ConfigurationProperties binds a whole prefix.
- Environment variables and command-line args override application.properties values.
- Use ${MY_VAR:default} to supply a fallback if the property is missing.
- Spring Boot loads 17 property sources in priority order -- app.properties is near the bottom.

Q1. Where does Spring Boot look for application.properties?

By default in src/main/resources/. You can also place it in the working directory or pass --spring.config.location on the command line.

Q2. Which has higher priority: application.properties or an environment variable?

Environment variables and command-line arguments override application.properties.

Q3. What is @ConfigurationProperties used for?

It binds all properties under a given prefix to the fields of a Java class, avoiding many separate @Value annotations.

Q4. How do you provide a default value with @Value?

Use the colon syntax: @Value("${app.name:MyApp}") returns 'MyApp' if app.name is not set.

Q5. Can you use both .properties and .yml at the same time?

Yes, but it can be confusing. If both exist, properties file takes precedence in Spring Boot 2; in Boot 3 both are loaded and merged.

5 **Profiles**
@Profile & spring.profiles.active for environment switching

What are Profiles?

Profiles let you have **different configurations for different environments** (dev, test, prod) without changing code. You create profile-specific property files and annotate beans or configuration classes with **@Profile**. Only the matching beans and properties are loaded.

Profile-specific property files

```
# application-dev.properties
server.port=8080
spring.datasource.url=jdbc:h2:mem:devdb

# application-prod.properties
server.port=443
spring.datasource.url=jdbc:mysql://prod-host:3306/proddb
```

@Profile on a bean

```
@Configuration
public class DataSourceConfig {

    @Bean
    @Profile("dev")
    public DataSource devDataSource() {
        return new EmbeddedDatabaseBuilder().setType(H2).build();
    }

    @Bean
    @Profile("prod")
    public DataSource prodDataSource() {
        return new HikariDataSource();  // real DB
    }
}
```

Activating a profile

```
# In application.properties
spring.profiles.active=dev

# Or on the command line
java -jar app.jar --spring.profiles.active=prod

# Or as environment variable
SPRING_PROFILES_ACTIVE=prod java -jar app.jar
```

Key Rules

- Name profile files as application-{profile}.properties or application-{profile}.yml.
- @Profile("!prod") means 'active on any profile EXCEPT prod'.
- Multiple profiles can be active at once: spring.profiles.active=dev,debug.
- spring.profiles.default sets the fallback if no profile is activated.
- In tests use @ActiveProfiles("test") instead of setting properties.

Q1. What is a Spring profile?

A named logical group of configuration. When a profile is active, only beans and properties tagged with that profile (or with no profile) are loaded.

Q2. How do you activate a profile in production?

Set the environment variable SPRING_PROFILES_ACTIVE=prod or pass --spring.profiles.active=prod as a JVM argument.

Q3. Can you have a bean that is active in all profiles except one?

Yes -- use @Profile("!prod") to exclude the bean when the 'prod' profile is active.

Q4. What is the default profile?

The 'default' profile. Beans with no @Profile annotation and properties in application.properties are always loaded regardless of active profiles.

Q5. How do you test a specific profile?

Annotate the test class with @ActiveProfiles("test") and create application-test.properties.

6 Spring Boot Actuator

Production-ready health, info, and metrics endpoints

What is Actuator?

Spring Boot Actuator adds **production-ready features** to your application with a single dependency. It exposes HTTP endpoints (and JMX) that let you monitor and manage the application at runtime -- check health, view metrics, inspect beans, change log levels, and more. All endpoints are secured by default and must be explicitly enabled.

Add the dependency

```
<!-- pom.xml -->
&lt;dependency>
    &lt;groupId>org.springframework.boot&lt;/groupId>
    &lt;artifactId>spring-boot-starter-actuator&lt;/artifactId>
&lt;/dependency>
```

Enable endpoints in application.properties

```
# Expose all endpoints over HTTP
management.endpoints.web.exposure.include=*

# Or expose only specific ones
management.endpoints.web.exposure.include=health,info,metrics

# Change the base path (default is /actuator)
management.endpoints.web.base-path=/manage

# Show detailed health info
management.endpoint.health.show-details=always
```

Actuator Endpoints Cheat Sheet

Endpoint	URL	What it shows
health	/actuator/health	App health status (UP/DOWN)
info	/actuator/info	App info (version, description)
metrics	/actuator/metrics	JVM, CPU, memory metrics
env	/actuator/env	Environment properties
beans	/actuator/beans	All Spring beans registered
mappings	/actuator/mappings	All @RequestMapping routes
loggers	/actuator/loggers	Logger levels (runtime change)
threaddump	/actuator/threaddump	Current thread states

Key Rules

- Add spring-boot-starter-actuator to pom.xml to enable Actuator.
- By default only /actuator/health and /actuator/info are exposed over HTTP.

- Use management.endpoints.web.exposure.include=* carefully in production.
- Secure Actuator endpoints with Spring Security to prevent unauthorized access.
- /actuator/health returns {status: UP} when the application is healthy.

Q1. What is Spring Boot Actuator?

A set of production-ready features (HTTP endpoints) that let you monitor and manage a Spring Boot application without adding custom code.

Q2. Which Actuator endpoints are enabled by default?

All endpoints are enabled (in-process) but only /health and /info are exposed over HTTP by default.

Q3. How do you expose the metrics endpoint?

Add management.endpoints.web.exposure.include=metrics (or *) to application.properties.

Q4. How can you secure Actuator endpoints?

Add spring-boot-starter-security and configure HttpSecurity to require authentication for paths starting with /actuator/.

Q5. What does /actuator/health return?

A JSON response like {"status":"UP"} by default. Set show-details=always to include database, disk space, and other health indicators.

7 Custom Auto-configuration

Write your own starter with @Conditional registration

Why Write a Custom Auto-configuration?

When you want to package reusable infrastructure (a custom HTTP client, a metrics reporter, a connection pool) as a **Spring Boot starter** library, you need to create a custom auto-configuration. The consuming application just adds your starter to pom.xml and everything is wired up automatically -- exactly like official Spring starters.

Step 1 -- Write the auto-configuration class

```
@AutoConfiguration
@ConditionalOnClass(GreetingService.class)
@ConditionalOnMissingBean(GreetingService.class)
@EnableConfigurationProperties(GreetingProperties.class)
public class GreetingAutoConfiguration {

    @Bean
    public GreetingService greetingService(GreetingProperties p) {
        return new GreetingService(p.getMessage());
    }
}
```

Step 2 -- Define configuration properties

```
@ConfigurationProperties(prefix = "greeting")
public class GreetingProperties {
    private String message = "Hello!";

    public String getMessage() { return message; }
    public void setMessage(String m) { this.message = m; }
}
```

Step 3 -- Register in AutoConfiguration.imports (Spring Boot 3)

```
# src/main/resources/META-INF/spring/
# org.springframework.boot.autoconfigure.AutoConfiguration.imports

com.example.greeting.GreetingAutoConfiguration

# Spring Boot 2 equivalent (spring.factories):
# org.springframework.boot.autoconfigure.EnableAutoConfiguration=\
# com.example.greeting.GreetingAutoConfiguration
```

Step 4 -- Consumer app just sets the property

```
# application.properties in consumer app
greeting.message=Hi from custom starter!

# Inject and use
@Autowired GreetingService gs;
// gs.greet() -> "Hi from custom starter!"
```

Key Rules

- Use @AutoConfiguration (Boot 3) instead of @Configuration for auto-config classes.

- Always add @ConditionalOnMissingBean so users can override your default bean.
- Register your class in META-INF/spring/...AutoConfiguration.imports (Boot 3).
- @EnableConfigurationProperties links your @ConfigurationProperties class.
- Name your starter artifact my-feature-spring-boot-starter by convention.

Q1. What is a Spring Boot starter?

A dependency JAR that bundles auto-configuration + required dependencies. Adding it to pom.xml is enough to activate a feature.

Q2. What annotation should a custom auto-configuration class use in Spring Boot 3?

@AutoConfiguration -- it is a specialization of @Configuration with ordering semantics.

Q3. Why use @ConditionalOnMissingBean in auto-configuration?

So the user's own bean takes precedence. If they declare the same type, your auto-configured bean is skipped.

Q4. Where do you register a custom auto-configuration in Spring Boot 3?

In META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports inside your library JAR.

Q5. Can two starters conflict?

Yes, if both provide a bean of the same type. @ConditionalOnMissingBean on each prevents conflicts -- only the first one found registers its bean.